# M04: Working with Files

> "The server is acting strange. The answer is buried in the logs — 50,000 lines deep. And once you find it, you'll need to fix the config, back it up, and set up a shortcut so it never takes this long again."

# Reading Files: cat, head, tail

`cat` — print an entire file to the terminal

```
$ cat /etc/hostname
tuwaiq-server
$ cat -n /etc/passwd     # add line numbers
```

`head` **and** `tail` — see the edges without loading everything

```
$ head -5 data.csv          # first 5 lines
$ tail -20 /var/log/syslog  # last 20 lines (most recent events)
```

**The debugging superpower:**

```
$ tail -f /var/log/syslog
```

- Stays running and prints **new lines live** as they are written — `Ctrl+C` to stop

> Production alert. Container won't start. First move: `tail -f /var/log/app.log`. This is reflex.

# Browsing Large Files: less

- **Opens a file in a scrollable viewer** — doesn't load the whole file into memory
  - `less /var/log/syslog` — browse a massive log without breaking a sweat

| Key | Action |
|---|---|
| `Space` / `b` | Page forward / backward |
| `/pattern` then `n` | Search forward, jump to next match |
| `q` | Quit |

  - `wc -l file` — counts lines before you open it: `wc -l /var/log/syslog` → "52,417 lines"
  - `file filename` — reveals what a file really is before you `cat` it: extensions are optional in Linux

> Remember using `man ls` in M02? That was `less` all along — same keys, same viewer.

# Finding Patterns: grep

`grep` filters a file to show **only matching lines**:

```
$ grep "error" /var/log/syslog       # lines containing "error"
$ grep -i "failed" /var/log/auth.log # case-insensitive
$ grep -n "timeout" app.log          # show line numbers
$ grep -c "warning" /var/log/syslog  # count matches only
```

**Search recursively through directories:**

```
$ grep -r "API_KEY" /etc/
```

**Show context around matches:**

```
$ grep -A 3 -B 2 "exception" app.log
```

`-A 3` = 3 lines After the match, `-B 2` = 2 lines Before — see what led up to the error

> In M03, `find` located files by name. `grep` finds files by **content**. Together, they're how real investigations work.

# The Three Streams

Every Linux command has **three data channels**:

| Stream | # | What It Carries |
|---|---|---|
| **stdin** | 0 | Data flowing **in** (keyboard or piped data) |
| **stdout** | 1 | Normal **results** (what you see on screen) |
| **stderr** | 2 | **Errors** and warnings (separate channel, same screen by default) |

- By default: stdin = keyboard, stdout + stderr both go to your terminal
- The `#` numbers matter when you want to reroute specific streams

> stdout and stderr look identical in the terminal — but they travel on separate wires. That's why you can redirect them independently.

# Pipes and Redirection

**Pipes** (`|`) connect commands — stdout of one becomes stdin of the next:

```
$ grep "failed" /var/log/auth.log | wc -l
23
```

**Redirection** connects commands to files:

| Operator | What It Does |
|---|---|
| `>` | Write stdout to a file (**overwrites**) |
| `>>` | Append stdout to a file (**adds to end**) |
| `2>` | Write **stderr** (errors) to a file |
| `2>/dev/null` | **Discard errors** — send them to a black hole |

```
$ find /etc -name "*.conf" -mtime -1 2>/dev/null
```

- `/dev/null` is a special device file that discards anything written to it
- `2>/dev/null` means: "send error messages (stream 2) to the discard device"
- Used when "Permission denied" noise would hide real results

> **The overwrite trap:** `>` destroys existing content. No undo. No recycle bin. When in doubt, use `>>`.

# Creating Files and Directories

`touch` — create empty files (or update timestamps)

```
$ touch README.md config.yaml app.js
```

`mkdir` — create directories (`-p` builds the full path)

```
$ mkdir -p webapp/{src,config,scripts,logs}
```

- Without `-p`: fails if the parent doesn't exist
- With `-p`: builds every level safely, even if some already exist
- The `{a,b,c}` brace expansion creates multiple directories in one command

> Four seconds, two commands. Your team lead said "set up the project."

# Copying and Moving Files

`cp` — copy files and directories

```
$ cp config.yaml config.yaml.bak       # backup before touching it
$ cp -r webapp/ webapp-backup/          # -r required for directories
```

`mv` — move OR rename (Linux treats renaming as a move)

```
$ mv draft.txt final-report.txt         # rename
$ mv config.yaml /etc/app/             # move to new location
```

- `cp` **needs** `-r` for directories — `mv` does **not**
- Both **silently overwrite** by default — add `-i` to get a confirmation prompt

> Always back up a config before editing it: `cp file file.bak`. That habit saves careers.

# Deleting Files and Directories

`rm` — delete permanently (**no trash can, no undo**)

```
$ rm old-notes.txt              # delete a file
$ rm -r old-project/            # delete a directory and everything inside
$ rm -ri dirname/               # interactive — prompts before each deletion
```

`rmdir` — only works on **empty** directories (safe choice)

- **Always** `ls` **first** to preview what you're about to destroy

| Command | What Happens |
|---|---|
| `rm -rf /` | Attempts to delete the **entire filesystem** |
| `rm -rf *` | Deletes everything in the current directory instantly |

> Pause. Preview. Then delete. This is the one command that can end your day early.

# Hard Links vs. Soft Links

Linux has two kinds of links — they look similar but behave very differently:

**Soft link (symlink)** — `ln -s` — a shortcut that points to another file by path

```
$ ln -s /opt/deployments/webapp/logs ~/quick-logs
$ ls -l ~/quick-logs
lrwxrwxrwx ... quick-logs -> /opt/deployments/webapp/logs
```

- The `l` at the start and `->` arrow identify a symlink in `ls -l` output
- Deleting the symlink never affects the original
- If the **target is deleted**, the symlink becomes **broken** (dangling pointer)

**Hard link** — `ln` (no `-s`) — another name for the exact same file

```
$ ln config.yaml config-hardlink.yaml
```

- Both names point to the same data on disk (same **inode**)
- Deleting one name leaves the data intact — the file is only gone when **all names are removed**
- Hard links cannot cross filesystems or link to directories

> Use soft links for shortcuts and cross-filesystem references. Use hard links when you want a true "alias" that can't become dangling.

# Analyzing Data: sort and uniq

`sort` — reorder lines

```
$ sort names.txt            # A-Z
$ sort -n scores.txt        # numeric (9 < 10, not 9 > 100)
$ sort -rn scores.txt       # highest first
```

`uniq` — remove adjacent duplicates (always sort first!)

```
$ sort names.txt | uniq -c  # count how many times each line appears
```

**The power pipeline — "top 10 most common X":**

```
$ sort access.log | uniq -c | sort -rn | head
    847 192.168.1.105
    623 10.0.0.44
```

`sort` → groups identical lines → `uniq -c` → counts each → `sort -rn` → ranks by count → `head` → top 10

> Memorize this pipeline. It's how cloud engineers analyze logs, audit traffic, and diagnose outages — with zero extra tools installed.

# Aha! Moment 💡

And: **"Every Terraform apply, every CI/CD pipeline — they're running these exact commands behind the scenes."**

What you just built:

- Read a 50,000-line log and found 12 relevant lines — instantly
- Created a full directory structure with one command
- Backed up configs before touching them
- Set up shortcuts to deeply nested paths

Real-world scenario:

- Production alert at 3 AM: "API timing out"
- You: `tail -f /var/log/app.log | grep "timeout"`
- Watch errors appear live as they happen
- Fix it. Back to bed.

> grep is the most-used debugging tool on the planet. Not an IDE. Not a GUI. Pattern matching, done right.

# Quick Reference

**Reading:**

| Command | Best For |
|---|---|
| `cat -n file` | Small files, with line numbers |
| `less file` | Large files — scroll, search, quit with `q` |
| `head -N` / `tail -N` | First/last N lines |
| `tail -f file` | Live monitoring |
| `wc -l file` | Count lines |
| `grep "pattern" file` | Find matching lines |

**Pipes and redirection:**

```
cmd1 | cmd2                # pipe stdout to next command
cmd > file                 # write to file (overwrite)
cmd >> file                # append to file
cmd 2>/dev/null            # discard error output
```

**File operations:**

```
touch file                 # create empty file
mkdir -p path/{a,b,c}      # create nested directories
cp -r src/ dest/           # copy (recursive for dirs)
mv old new                 # move or rename
rm -ri dirname/            # interactive delete (safer)
ln -s /target linkname     # soft link (shortcut)
ln file hardlink           # hard link (same inode)
sort data | uniq -c | sort -rn | head   # top 10 pattern
```

# Mission Complete: Working with Files ✅

**You can now read, search, create, organize, and analyze any file on the system.**

- `cat` / `less` / `head` / `tail` — read files of any size
- `grep` — find exactly what you need in thousands of lines
- **Streams + pipes + redirection** — chain commands and route data anywhere
- `touch` / `mkdir` / `cp` / `mv` / `rm` — build and manage the filesystem
- **Soft links** point by path (can break); **hard links** share an inode (cannot break)
- `sort | uniq -c | sort -rn` — the universal log analysis pattern

> Next mission: you can read and edit configs — but you need a real editor to do it effectively. That's the editor's toolkit.

# Questions?